MONAD BLITZ HACKATHON · MVP GUIDE

Reflex

Multi-Player Reaction-Time Wagering Game on Monad. Panduan lengkap dari smart contract sampai frontend, untuk dikerjakan 2 orang dalam 2 jam.

DURASI	TIM	STACK
2 jam	**2 developer**	**Foundry · Next.js · viem**

Daftar Isi

1. Konsep & Justifikasi Monad
2. Arsitektur Sistem
3. Smart Contract — Reflex.sol Multi-player
4. Game Flow Multi-player
5. Tantangan Sync N Device
6. Pembagian Kerja 2 Orang × 2 Jam
7. Demo Script untuk Panggung
8. Cut-list Kalau Mepet
9. Catatan Teknis Tambahan

1. Konsep & Justifikasi Monad

Game dalam 1 Kalimat

N pemain stake MON → countdown random 1-5 detik → "GO!" muncul → pemain dengan reaction time tercepat menang seluruh pot.

Kenapa Ide Ini Lulus Filter "Mati di Ethereum, Hidup di Monad"

- **Reaction time matter dalam millidetik.** Kalau finality 12 detik (Ethereum), tap pemenang baru confirmed setelah game lain selesai. Demo flat.
- **Monad finality 800ms** bikin gameplay terasa instant. Pemenang langsung ketahuan.
- Aligned dengan thesis Monad: *"skill-based risk-to-earn interfaces where players can wager on their own outcomes — at an atomic and micro level, allowing thousands of competitions to be run."*

Kenapa Demoable di Panggung

- 2-10 audience volunteer per match, 30 detik per match, 5 match = ramai dan kompetitif
- Visual sederhana, paham dalam 5 detik
- Pemenang dapat MON real, bisa dilihat di wallet mereka langsung
- Bisa dijalankan multi-round = audience engagement total

2. Arsitektur Sistem

```
┌──────────────────┐          ┌──────────────────┐
│   Frontend       │ ──────►  │  Smart Contract  │
│  (Next.js)       │          │  (Monad Testnet) │
│                  │ ◄──────  │                  │
└──────────────────┘  events  └──────────────────┘
         │
         │
         │ poll/listen via viem watchContractEvent
         ▼
┌──────────────────────────┐
│ In-app state (no BE)     │
│ Read events directly     │
└──────────────────────────┘
```

BACKEND & INDEXER?

Tidak perlu backend. Frontend langsung baca contract via RPC. Untuk MVP 2 jam, **skip Envio HyperIndex** juga — pakai event log direct read via `getLogs` dan `watchContractEvent` sudah cukup. Envio bagus untuk versi production, tapi makan waktu setup yang tidak worth dalam timebox 2 jam.

Stack Final

Komponen	Tools	Catatan
Smart contract	Foundry, Solidity 0.8.20	Cepat compile + test
Frontend	Next.js 14 + TypeScript	App Router
Web3	wagmi + viem	Set `pollingInterval: 200ms`
Auth	Privy embedded wallet	Auto-sign, no popup
Styling	Tailwind	Cepat untuk demo polish
State	React useState	Jangan Redux untuk 2 jam
Indexer	**Skip**	Event log direct read

3. Smart Contract — Reflex.sol Multi-Player

Berikut contract lengkap yang mendukung 2-20 pemain per match, dengan host control dan auto-settle.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract Reflex {
    enum MatchState { Open, Locked, Active, Finished }

    struct Match {
        address host;
        uint256 stakePerPlayer;
        uint8 maxPlayers;
        address[] players;
        uint256 goTimestampMs;
        uint256 settleDeadlineMs;
        address winner;
        uint256 winnerReactionMs;
        uint8 tappedCount;
        MatchState state;
    }

    mapping(uint256 => Match) public matches;
    mapping(uint256 => mapping(address => bool)) public hasTapped;
    mapping(address => uint256) public bestReactionMs;

    uint256 public matchCounter;
    uint8 public constant MIN_PLAYERS = 2;
    uint8 public constant MAX_PLAYERS_CAP = 20;

    event MatchCreated(uint256 indexed matchId, address host, uint256 stake, uint8 maxPlayers);
    event MatchJoined(uint256 indexed matchId, address player, uint8 currentCount);
    event MatchStarted(uint256 indexed matchId, uint256 goTimestampMs, uint8 totalPlayers);
    event TapSubmitted(uint256 indexed matchId, address player, uint256 reactionMs);
    event MatchFinished(uint256 indexed matchId, address winner, uint256 reactionMs, uint256 pot

    function createMatch(uint8 maxPlayers) external payable returns (uint256) {
        require(msg.value > 0, "Stake required");
        require(maxPlayers >= MIN_PLAYERS && maxPlayers <= MAX_PLAYERS_CAP, "Invalid count");

        matchCounter++;
        Match storage m = matches[matchCounter];
        m.host = msg.sender;
        m.stakePerPlayer = msg.value;
        m.maxPlayers = maxPlayers;
        m.state = MatchState.Open;
        m.players.push(msg.sender);

        emit MatchCreated(matchCounter, msg.sender, msg.value, maxPlayers);
        emit MatchJoined(matchCounter, msg.sender, 1);
        return matchCounter;
    }

    function joinMatch(uint256 matchId) external payable {
        Match storage m = matches[matchId];
        require(m.state == MatchState.Open, "Not joinable");
        require(msg.value == m.stakePerPlayer, "Stake mismatch");
        require(m.players.length < m.maxPlayers, "Match full");
```

```
        for (uint i = 0; i < m.players.length; i++) {
            require(m.players[i] != msg.sender, "Already joined");
        }

        m.players.push(msg.sender);
        emit MatchJoined(matchId, msg.sender, uint8(m.players.length));

        if (m.players.length == m.maxPlayers) {
            m.state = MatchState.Locked;
        }
    }

    function lockMatch(uint256 matchId) external {
        Match storage m = matches[matchId];
        require(msg.sender == m.host, "Only host");
        require(m.state == MatchState.Open, "Not open");
        require(m.players.length >= MIN_PLAYERS, "Need 2+ players");
        m.state = MatchState.Locked;
    }

    function startMatch(uint256 matchId) external {
        Match storage m = matches[matchId];
        require(msg.sender == m.host, "Only host");
        require(m.state == MatchState.Locked, "Not locked");

        m.goTimestampMs = block.timestamp * 1000;
        m.settleDeadlineMs = m.goTimestampMs + 5000;
        m.state = MatchState.Active;

        emit MatchStarted(matchId, m.goTimestampMs, uint8(m.players.length));
    }

    function submitTap(uint256 matchId, uint256 clientTimestampMs) external {
        Match storage m = matches[matchId];
        require(m.state == MatchState.Active, "Not active");
        require(!hasTapped[matchId][msg.sender], "Already tapped");
        require(_isPlayer(m, msg.sender), "Not a player");
        require(clientTimestampMs > m.goTimestampMs, "Too early");
        require(clientTimestampMs <= m.settleDeadlineMs, "Too late");

        uint256 reactionMs = clientTimestampMs - m.goTimestampMs;
        hasTapped[matchId][msg.sender] = true;
        m.tappedCount++;

        if (m.winner == address(0) || reactionMs < m.winnerReactionMs) {
            m.winner = msg.sender;
            m.winnerReactionMs = reactionMs;
        }

        if (bestReactionMs[msg.sender] == 0 || reactionMs < bestReactionMs[msg.sender]) {
            bestReactionMs[msg.sender] = reactionMs;
        }

        emit TapSubmitted(matchId, msg.sender, reactionMs);

        if (m.tappedCount == m.players.length) {
            _settle(matchId);
```

```
        }
    }

    function settleMatch(uint256 matchId) external {
        Match storage m = matches[matchId];
        require(m.state == MatchState.Active, "Not active");
        require(block.timestamp * 1000 > m.settleDeadlineMs, "Too early");
        require(m.winner != address(0), "No tap submitted");
        _settle(matchId);
    }

    function _settle(uint256 matchId) internal {
        Match storage m = matches[matchId];
        m.state = MatchState.Finished;

        uint256 pot = m.stakePerPlayer * m.players.length;
        (bool sent, ) = m.winner.call{value: pot}("");
        require(sent, "Payout failed");

        emit MatchFinished(matchId, m.winner, m.winnerReactionMs, pot);
    }

    function _isPlayer(Match storage m, address addr) internal view returns (bool) {
        for (uint i = 0; i < m.players.length; i++) {
            if (m.players[i] == addr) return true;
        }
        return false;
    }

    function getMatchPlayers(uint256 matchId) external view returns (address[] memory) {
        return matches[matchId].players;
    }
}
```

Highlight Desain

Aspek	Pendekatan
State Machine	`Open → Locked → Active → Finished` — Locked penting agar tidak ada pemain join saat host trigger countdown
Anti-cheat	Trust client timestamp (cukup untuk demo). Production: pakai VRF + commit-reveal
Random delay	Di-handle frontend, bukan contract. Contract hanya tahu kapan `startMatch` dipanggil
Auto-settle	Saat semua pemain tap, atau manual settle setelah deadline (handle pemain afk)
Host control	Hanya host bisa lock dan start (cegah trolling)
Gas	Loop O(N) untuk join check — OK untuk N≤20

4. Game Flow Multi-player

Step 1 — Host membuat match

`createMatch(maxPlayers=10)` + stake 0.01 MON → `matchId = 5`

Step 2 — Share link

Host share link `/match/5` ke audience (via QR code di slide)

Step 3 — Players join

Audience tap link, panggil `joinMatch(5)` + stake. Event `MatchJoined` emit, semua frontend update count "3/10"

Step 4 — Host start game

Host klik "Start Game" → `lockMatch(5)` [optional kalau belum full] → frontend countdown random 1-5 detik visual ("WAIT...") → `startMatch(5)`

Step 5 — TAP race

Event `MatchStarted` masuk → semua frontend tampilkan "TAP NOW!" hijau full-screen → players tap → `submitTap(5, Date.now())`

Step 6 — Auto-settle

Setelah semua tap atau timeout 5 detik → event `MatchFinished` → pemenang dapat pot

5. Tantangan Sync N Device "GO!" Bersamaan

Ini bagian paling tricky di multi-player. Ada 3 pendekatan:

Pendekatan A — Pure on-chain

Frontend tunggu event `MatchStarted` → tampilkan "GO!". Karena Monad finality 800ms, semua device dapat event dalam 800ms range. Beda 200-300ms antar device acceptable karena measurement reaction time mulai dari `goTimestampMs` yang sama.

Pendekatan B — Hybrid RECOMMENDED

Host trigger `startMatch()`. Frontend semua pemain listen event. Saat event masuk, hitung reaction time dari `event.goTimestampMs` (bukan `Date.now()` saat dilihat di layar). **Fairness sempurna** karena baseline waktu sama untuk semua.

Pendekatan C — Off-chain coordination

Pakai WebSocket/Pusher untuk broadcast "GO!". Lebih cepat tapi butuh server. **Skip untuk 2 jam.**

IMPLEMENTASI PENDEKATAN B

- Frontend listen `MatchStarted` via `watchContractEvent` dengan `pollingInterval: 200`
- Saat event masuk, save `goTimestampMs` di state
- User tap → `submitTap(matchId, Date.now())`
- Reaction time dihitung di contract: `clientTimestampMs - goTimestampMs`

6. Pembagian Kerja 2 Orang × 2 Jam

Orang A

Smart Contract + Web3 Layer

Solidity, Foundry, hooks viem, helpers ABI

Orang B

Frontend UI + Game Loop

Next.js, screens, multi-player UX

Jam 0:00 — 0:15 · Setup Bareng

- Setup repo monorepo: `/contracts` + `/web`
- Tentukan testnet RPC + faucet (Monad testnet, chain ID 10143)
- Tukar Discord/Telegram untuk komunikasi cepat

```
reflex/
├── contracts/        # Foundry project (Orang A)
│   └── src/Reflex.sol
└── web/              # Next.js app (Orang B)
    └── app/page.tsx
```

Jam 0:15 — 1:00 · Parallel Work (45 menit)

Orang A — Smart Contract

- (20 min) Setup Foundry, copy contract, sesuaikan
- (15 min) Test 3-player scenario di Foundry
- (10 min) Deploy ke Monad testnet, share address + ABI

Orang B — Frontend Skeleton

- (15 min) `create-next-app`, install wagmi + viem + Privy
- (15 min) Setup wagmi config Monad testnet, Privy embedded wallet
- (15 min) Buat 3 route: `/`, `/create`, `/match/[id]` + komponen `PlayerList`

```
# Deploy command (Orang A)
forge create src/Reflex.sol:Reflex \
  --rpc-url $MONAD_RPC \
  --private-key $PK
```

Jam 1:00 — 1:30 · Integration + Game Loop

Orang A — Hooks & Helpers

- (15 min) `useMatch(matchId)` hook
- (15 min) `useMatchEvents()` — listen 4 event dengan `pollingInterval: 200ms`

Orang B — Match Screen Logic

- Phase Open: player list + tombol Join. Host lihat Lock + Start
- Phase Locked: countdown random 1-5 detik visual
- Phase Active: flash hijau "TAP!" → submitTap
- Phase Finished: tampilkan winner + reaction time semua

Jam 1:30 — 1:50 · Polish + Live Leaderboard

Orang A — In-Match Leaderboard

(20 min) Tiap event `TapSubmitted` masuk, update list real-time. Sort by reaction time. Tampilkan di bawah hasil.

Orang B — Demo Polish

- Big visual saat "TAP!" — full screen hijau, font besar
- QR code di lobby supaya audience scan + join cepat
- Display "5/10 joined" yang update real-time

Jam 1:50 — 2:00 · E2E Test + Demo Prep

- Test dengan 3 device (HP teman + laptop + tablet)
- Pastikan: join → start → semua dapat "TAP!" sync → leaderboard urut benar
- Backup: pre-create 1 match dengan stake kecil supaya demo cepat tinggal share QR

7. Demo Script untuk Panggung

> ***"Saya butuh 10 volunteer. Buka kamera HP, scan QR ini."***
>
> *[QR di slide → semua scan, masuk ke /match/5]*
>
> *[Layar besar tampilkan player list naik 1...3...7...10]*
>
> ***"Semua sudah masuk. Saya start game dalam 3-2-1..."***
>
> *[Host klik Start → countdown random 2-5 detik, semua HP tampilkan 'GET READY...']*
>
> *[BAM — semua HP flash hijau bersamaan, "TAP!"]*
>
> *[10 reaction time muncul real-time di layar besar, ranked]*
>
> ***"Pemenang: 0xabc...def dengan 187ms. Pot 0.1 MON sudah masuk wallet kalian. Settle ke chain dalam < 1 detik. Coba lakukan ini di Polygon."***

Yang HARUS Jalan Saat Demo

1. **Host buat match dari panggung** — share QR di slide

2. **Minimal 5+ audience join** dalam < 30 detik
3. **Lock + start trigger** smooth, countdown visible
4. **Semua device dapat "TAP!"** hampir bersamaan (delta < 1 detik OK)
5. **Pemenang muncul instant** dengan reaction time semua pemain ranked

8. Cut-list Kalau Mepet

> **TRIAGE ORDER**
> Kalau Jam 1:30 ternyata gameplay belum jalan, **CUT** dalam urutan ini:

1. **Personal best leaderboard global** — fokus per-match dulu
2. **QR code generator** — kasih link manual, audience copy paste
3. **Manual lock** — auto-lock saat penuh saja, host tinggal nunggu N orang
4. **Stake variabel** — hard-code 0.001 MON
5. **Privy embedded wallet** — pakai injected MetaMask saja kalau lebih cepat setup

9. Catatan Teknis Tambahan

Hal yang Sering Bikin Gagal di 2 Jam

Masalah	Solusi
Wallet connection issue	Pakai Privy lebih cepat dari WalletConnect; pakai 1 metode saja
RPC rate limit	Pakai dedicated RPC (Alchemy free tier untuk Monad), jangan public RPC
Event listening lambat	Set `pollingInterval: 500` (atau 200) di config viem — default 4s terlalu lambat
Race condition tap	Untuk N-player: contract handle "fastest wins", bukan "first tx wins" — tidak masalah
Storage gas	`players[]` push ~20k gas. OK untuk N≤20. Jangan 100+.

RPC Load untuk N Device

N device polling event setiap 200ms = N×5 req/sec. Untuk N=10, itu 50 req/sec. **Pakai dedicated RPC** (Alchemy/Chainstack), free tier cukup.

Variabel Lingkungan yang Diperlukan

```
# .env (root)
MONAD_RPC=https://testnet-rpc.monad.xyz
PRIVATE_KEY=0x...

# web/.env.local
NEXT_PUBLIC_CONTRACT_ADDRESS=0x...
NEXT_PUBLIC_RPC_URL=https://...
NEXT_PUBLIC_PRIVY_APP_ID=...
```

Ringkasan Filter "Mati di Ethereum"

WHY REFLEX LULUS TES

- Reaction time matter dalam millidetik — finality 12s = mustahil
- Multi-player simultaneous tap = burst TPS yang jelas terlihat
- Settle & payout instant = UX yang feels native, bukan "wait 30 detik"
- Tournament mode (extension) bisa generate 1000+ tx dalam 10 menit demo

---

Dibuat untuk Monad Blitz Hackathon · Ship fast, ship onchain.